

Cloud-Native and Multi-Model Databases

Chapter 06 - Sub-Chapter 10

NoSQL Databases Series

Cloud-Native Landscape	Managed services by category across all clouds
Azure Cosmos DB	Multi-model, 5 APIs, global active-active
Aurora and Cloud Spanner	NewSQL: SQL at NoSQL scale
Serverless Patterns	PlanetScale, Neon, CDC, database branching

1 Cloud-Native Databases

Managed, elastic, globally distributed

In the past, running a database meant buying servers, installing software, configuring replication, setting up backups, and hiring DBAs to manage it all. Today, every major cloud provider offers fully managed database services that handle all of this automatically.

Cloud-native does not just mean 'running in the cloud' — it means designed from the ground up for elastic scalability, high availability, pay-per-use billing, and deep integration with other cloud services like serverless functions, event streams, and identity management.

Cloud-Native Database Landscape — Managed Services by Category

Relational	Document	Wide-Column	Key-Value	Multi-Model
Aurora (AWS) Cloud SQL (GCP) Azure SQL	Atlas (MongoDB) Cosmos DB (Azure) Firestore (GCP)	Bigtable (GCP) Keyspaces (AWS) Cosmos Cassandra	DynamoDB (AWS) Redis Cloud ElastiCache	Cosmos DB FaunaDB YugabyteDB

Key advantages of cloud-native databases:

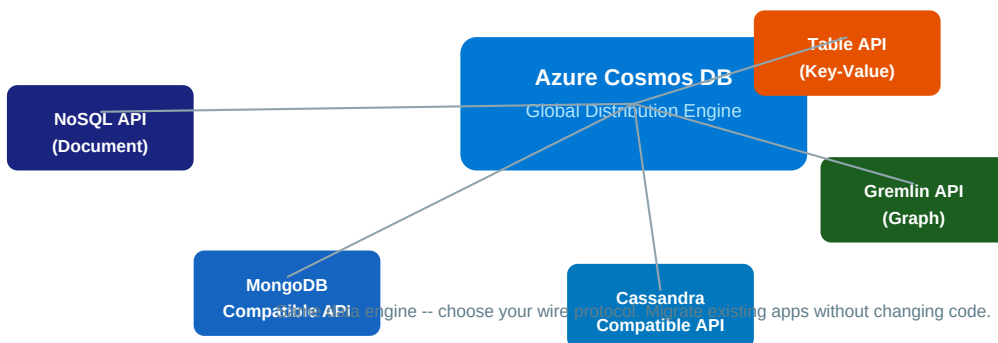
- **Zero operational overhead:** No patching, no replication setup, no backup scripts. The cloud manages everything.
- **Elastic scaling:** Scale from 0 to millions of requests without pre-provisioning capacity.
- **Global distribution:** Replicate to multiple regions with a checkbox. Sub-second global consistency.
- **Pay-per-use:** Pay only for what you consume — idle databases cost nothing (serverless tier).
- **Integration ecosystem:** Native triggers to Lambda/Cloud Functions, CDC streams, IAM authentication.
- **SLA guarantees:** Cloud providers guarantee 99.99%+ availability — hard to achieve self-hosted.

2 Azure Cosmos DB

The ultimate multi-model, globally distributed database

Cosmos DB is Microsoft Azure's flagship NoSQL database — and arguably the most ambitious database product ever built. It offers single-digit millisecond latency globally, 99.999% availability, and supports five different APIs (wire protocols) on the same underlying engine.

Azure Cosmos DB — One Engine, Many APIs



What makes Cosmos DB unique:

- **5 consistency levels:** Strong, Bounded Staleness, Session, Consistent Prefix, Eventual — fine-grained control.
- **Multi-master active-active:** Write to any region, reads and writes are always local.
- **Automatic indexing:** Every property is indexed by default — no schema or index management.
- **Wire-compatible APIs:** Bring your existing MongoDB or Cassandra code — point it at Cosmos DB.

Spring Boot with Cosmos DB (using Azure SDK):

```
<!-- pom.xml -->
<dependency>
  <groupId>com.azure.spring</groupId>
  <artifactId>spring-cloud-azure-starter-data-cosmos</artifactId>
  <version>5.10.0</version>
</dependency>
```

```
# application.yml
spring:
  cloud:
    azure:
      cosmos:
        endpoint: https://myaccount.documents.azure.com:443/
        key: ${COSMOS_PRIMARY_KEY}
        database: ecommerce
        populate-query-metrics: true
```

```
import com.azure.spring.data.cosmos.core.mapping.Container;
import com.azure.spring.data.cosmos.core.mapping.PartitionKey;
import com.azure.spring.data.cosmos.repository.CosmosRepository;
import org.springframework.stereotype.Repository;
import java.util.List;

@Container(containerName = "orders")    // Cosmos DB container
public class Order {
```

```
@Id
private String id;

@PartitionKey                      // partition key for distribution
private String userId;

private String status;
private double total;
// getters/setters
}

@Repository
public interface OrderRepository extends CosmosRepository<Order, String> {
    List<Order> findById(String userId);
    List<Order> findByIdAndStatus(String userId, String status);

    // Cosmos DB SQL query
    @Query("SELECT * FROM orders o WHERE o.total > @minTotal")
    List<Order> findExpensiveOrders(@Param("minTotal") double minTotal);
}
```

3 NewSQL — Aurora and Cloud Spanner

SQL semantics at NoSQL scale

For years, the trade-off was clear: SQL = consistency + limits on scale, NoSQL = scale + eventual consistency. NewSQL databases broke this trade-off — they offer full ACID SQL semantics with horizontal scalability. Two leading examples: Amazon Aurora and Google Cloud Spanner.

Amazon Aurora — MySQL/PostgreSQL compatible, 5x faster:

Aurora separates compute from storage. Storage is a distributed log replicated across 6 copies in 3 availability zones. Compute nodes (read replicas) can be added in minutes. Aurora Serverless v2 auto-scales compute in under a second — perfect for variable workloads.

```
# Aurora Serverless v2 -- Spring Boot connects like normal PostgreSQL
spring:
  datasource:
    url: jdbc:postgresql://mydb.cluster-xyz.us-east-1.rds.amazonaws.com:5432/ecommerce
    username: appuser
    password: ${RDS_PASSWORD}
    # Aurora auto-scales -- no capacity planning needed
    hikari:
      maximum-pool-size: 20

# Use IAM authentication instead of passwords (more secure)
spring:
  datasource:
    url: jdbc:postgresql://mydb.cluster-xyz.us-east-1.rds.amazonaws.com:5432/ecommerce?ssl=true
    # Aurora generates short-lived IAM tokens -- no stored passwords
```

Google Cloud Spanner — globally consistent SQL at planet scale:

Spanner is Google's internal database, open to developers since 2017. It achieves something previously considered impossible: external consistency (stronger than serializable) across multiple continents. It uses TrueTime — atomic clocks and GPS in every Google datacenter — to timestamp transactions with guaranteed global ordering.

```
import com.google.cloud.spanner.DatabaseClient;
import com.google.cloud.spanner.Mutation;
import com.google.cloud.spanner.ResultSet;
import com.google.cloud.spanner.Statement;
import java.util.Collections;

// Spanner Java client
DatabaseClient dbClient = spanner.getDatabaseClient(
    DatabaseId.of("my-project", "my-instance", "ecommerce")
);

// Write with Mutations (strongly typed)
dbClient.write(Collections.singletonList(
    Mutation.newInsertOrUpdateBuilder("orders")
        .set("order_id").to("ord-001")
        .set("user_id").to("alice")
        .set("status").to("CONFIRMED")
        .set("total").to(120.50)
        .build()
));

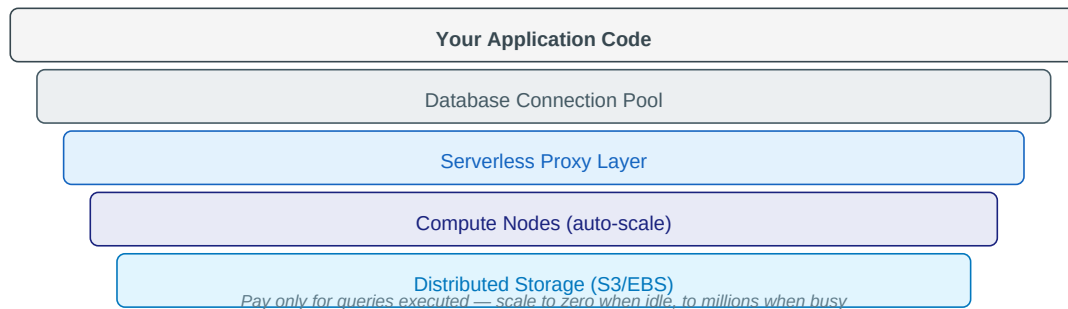
// Query with SQL -- full Google Standard SQL
```

```
try (ResultSet rs = dbClient.singleUse().executeQuery(
    Statement.of("SELECT o.order_id, u.name FROM orders o " +
        "JOIN users u ON o.user_id = u.user_id " +
        "WHERE o.status = 'CONFIRMED' ORDER BY o.created_at DESC LIMIT 10"))) {
    while (rs.next()) {
        System.out.println(rs.getString("order_id") + " - " + rs.getString("name"));
    }
}
```

4 Serverless Database Patterns

PlanetScale, Neon, FaunaDB, and best practices

Serverless Database Architecture



New generation of serverless databases:

Database	Type	Key Feature	Best For
PlanetScale	MySQL (serverless)	Schema branching like Git, zero-downtime migrations	Teams deploying frequently, schema changes
Neon	PostgreSQL (serverless)	Scale to zero, branch databases, instant preview envs	Dev/staging environments, cost-sensitive startups
FaunaDB	Multi-model	ACID globally, GraphQL native, serverless billing	Jamstack apps, global ACID without servers
CockroachDB	NewSQL (distributed PostgreSQL)	PostgreSQL wire-compatible, multi-region ACID	Global apps needing PostgreSQL consistency
TiDB	NewSQL (MySQL compat)	OLTP + OLAP in one, Spark-compatible	Hybrid operational/analytical workloads

Database branching — the future of schema management:

PlanetScale and Neon introduced a concept borrowed from Git: **database branches**. Just like you branch code to develop a feature, you can branch a database. Your feature branch gets its own schema copy — no risk to production.

```
# PlanetScale -- branch workflow (like Git for your database)
# Create a feature branch for a new schema change
pscale branch create mydb add-shipping-columns

# Apply schema changes on the branch (no downtime, no locking)
pscale shell mydb add-shipping-columns
# > ALTER TABLE orders ADD COLUMN shipping_method VARCHAR(50);

# Open a deploy request (like a pull request for schema)
pscale deploy-request create mydb add-shipping-columns
# Team reviews, approves, and merges schema change safely

# Spring Boot: point to branch for local dev, main for production
```

```
spring:
  datasource:
    url: jdbc:mysql://aws.connect.psdb.cloud/mydb?sslMode=VERIFY_IDENTITY
    # PlanetScale uses connection strings per branch
```

Change Data Capture (CDC) — database as an event stream:

Modern cloud databases support CDC — streaming every database change as an event. This enables real-time data synchronization, search index updates, and event-driven architectures without polling.

```
// DynamoDB Streams + Lambda (CDC pattern)
// Every change to DynamoDB triggers a Lambda function

// Lambda handler (triggered on DynamoDB stream events)
public class OrderStreamHandler implements RequestHandler<DynamodbEvent, Void> {

    @Override
    public Void handleRequest(DynamodbEvent event, Context context) {
        for (DynamodbStreamRecord record : event.getRecords()) {
            if ("INSERT".equals(record.getEventName())) {
                // New order placed -- trigger notification workflow
                Map<String, AttributeValue> newOrder = record.getDynamodb().getNewImage();
                String orderId = newOrder.get("order_id").getS();
                String userId = newOrder.get("user_id").getS();
                sendOrderConfirmationEmail(userId, orderId); // async, downstream
            }
            if ("MODIFY".equals(record.getEventName())) {
                // Order updated -- reindex in Elasticsearch
                reindexInElasticsearch(record.getDynamodb().getNewImage());
            }
        }
        return null;
    }
}
```

Key Rules

- Prefer managed cloud databases over self-hosted -- operational savings outweigh licensing costs.
- Cosmos DB: use when you need multi-region active-active with flexible data model on Azure.
- Aurora Serverless v2: use when you need PostgreSQL/MySQL but traffic is highly variable.
- Cloud Spanner: use when you need globally consistent SQL -- accept higher cost for correctness.
- CDC (DynamoDB Streams, MongoDB Change Streams): use to build event-driven microservices without polling.